

SIMATS ENGINEERING
Assignment Question

COURSE NAME: Data Base Management System

COURSE CODE: CSA0501

NAME: Mukesh Raj L

REG: 192571036

1)

An E-Commerce Order Management Database using
Customer(Customer_ID, Customer_Name, Email, Phone),
Product(Product_ID, SKU, Product_Name, Category_ID,
Unit_Price, Stock_Qty), Category(Category_ID,
Category_Name), Orders(Order_ID, Customer_ID,
Order_Date, Order_Status, Total_Amount),
Order_Item(Order_ID, Product_ID, Quantity,
Selling_Price), Payment(Payment_ID, Order_ID,
Payment_Mode, Amount, Payment_Status). Analyze
functional dependencies and normalize the relations. Write
SQL queries for top-selling products and customer-wise
sales. Create a sales summary view. Design indexes for
Customer_ID and Product_ID. Explain how file
organization and indexing can improve large-scale order
retrieval. (100 marks)

DATABASE MANAGEMENT SYSTEMS – PROJECT REPORT

1. Problem Statement

The proposed system is an E-Commerce Order Management Database that stores and manages customers, products, product categories, orders, order items and payments. The database must support reliable order processing as well as analytical queries such as identifying top-selling products and calculating customer-wise sales. The initial business data can be viewed as a collection of related facts: a customer places orders, each order contains one or more products, each product belongs to a category, and an order can have an associated payment. If these facts are stored in a single large relation, redundancy, update anomalies and insertion/deletion anomalies can occur. The project therefore analyzes functional dependencies and transforms the relations into a normalized relational design. SQL queries are developed for sales analysis, a sales summary view is created for repeated reporting, and indexes are designed on Customer_ID and Product_ID to improve joins and retrieval. File organization and indexing are also discussed in the context of large-scale order retrieval.

2. Objective

The main objective is to design a well-structured relational database for e-commerce order management and demonstrate how normalization, SQL analytics, views, indexing and file organization contribute to an efficient DBMS solution.

2.2 Expected Outcome

The main objective of this project is to design and develop a well-structured E-Commerce Order Management Database that can efficiently store and manage customer, product, category, order, order-item, and payment information. The project aims to analyze functional dependencies and normalize the relations up to Third Normal Form (3NF) to reduce data redundancy and maintain data consistency. It also focuses on implementing SQL queries to identify top-selling products and calculate customer-wise sales, creating a Sales_Summary view for reusable reporting, and designing indexes on Customer_ID and Product_ID to improve data retrieval and query performance. In addition, the project explains how file organization and indexing can support efficient retrieval of large volumes of order data.

3. Requirements and Environment Used

3.1 Hardware Requirements

Component	Recommended Requirement
Processor	Dual-core processor or above
RAM	4 GB minimum; 8 GB recommended
Storage	At least 2 GB free for DBMS, tools and project files
Display	1366 × 768 or higher

3.2 Software Requirements

Software	Purpose
MySQL 8.x	Relational database management system
MySQL Workbench	SQL development, execution and database visualization
Windows / Linux / macOS	Operating system
Web browser	Optional reference/documentation access
Microsoft Word / compatible editor	Report preparation

3.3 Database Objects

Object	Purpose
CUSTOMER	Stores customer master information
CATEGORY	Stores product category information
PRODUCT	Stores product master and stock information
ORDERS	Stores order header information
ORDER_ITEM	Stores individual products within an order
PAYMENT	Stores payment information
SALES_SUMMARY	View for customer/order sales reporting

4. Design / Proposed Solution

Proposed Relational ER Structure

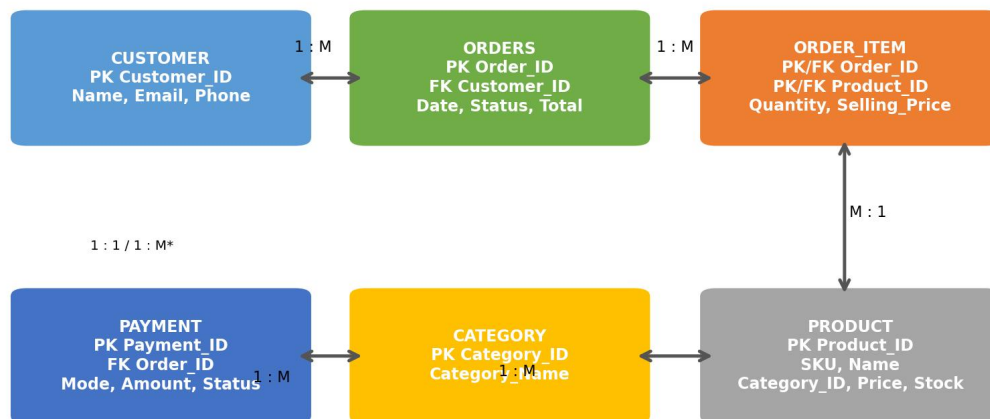


Figure 2. Relational ER-style structure showing the major entities and relationships.

4.1 Relational Schema

Relation	Attributes
CUSTOMER	Customer_ID (PK), Customer_Name, Email, Phone
CATEGORY	Category_ID (PK), Category_Name
PRODUCT	Product_ID (PK), SKU, Product_Name, Category_ID (FK), Unit_Price, Stock_Qty
ORDERS	Order_ID (PK), Customer_ID (FK), Order_Date, Order_Status, Total_Amount
ORDER_ITEM	Order_ID (PK/FK), Product_ID (PK/FK), Quantity, Selling_Price
PAYMENT	Payment_ID (PK), Order_ID (FK), Payment_Mode, Amount, Payment_Status

4.2 Relationship Description

- One customer can place many orders; therefore CUSTOMER to ORDERS is 1:M.
- One order contains many order items; therefore ORDERS to ORDER_ITEM is 1:M.
- One product can occur in many order items; therefore PRODUCT to ORDER_ITEM is 1:M.
- One category can contain many products; therefore CATEGORY to PRODUCT is 1:M.
- An order can have one or more payment records depending on the business rule; the design permits 1:M while a unique constraint can be added if exactly one payment is required.

4.3 Functional Dependencies

- Customer_ID → Customer_Name, Email, Phone
- Category_ID → Category_Name
- Product_ID → SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty
- SKU → Product_ID, Product_Name, Category_ID, Unit_Price, Stock_Qty (assuming SKU is unique)
- Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount
- (Order_ID, Product_ID) → Quantity, Selling_Price
- Payment_ID → Order_ID, Payment_Mode, Amount, Payment_Status

The dependencies show that each entity has attributes determined by its identifier. In ORDER_ITEM, the combination of Order_ID and Product_ID identifies a line item in the basic design, so Quantity and Selling_Price depend on the full composite key.

4.4 Normalization Analysis

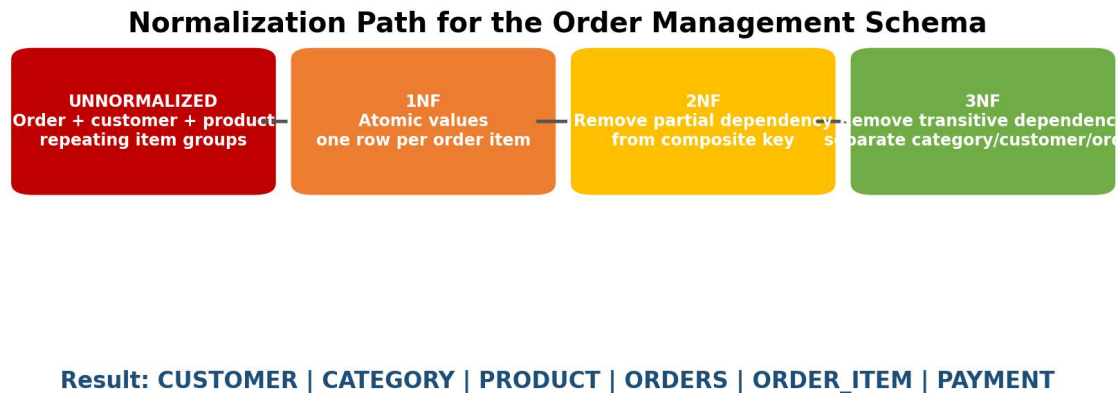


Figure 3. Normalization path from a combined order structure to a 3NF design.

First Normal Form (1NF)

A relation is in 1NF when attributes contain atomic values and repeating groups are removed. For order data, each Order_Item should represent one product occurrence with one Quantity and one Selling_Price. Product lists should not be stored as comma-separated values inside Orders.

Second Normal Form (2NF)

2NF requires 1NF and removal of partial dependencies on a proper subset of a composite candidate key. In ORDER_ITEM, Quantity and Selling_Price depend on the full combination (Order_ID, Product_ID), not on Order_ID alone or Product_ID alone.

Third Normal Form (3NF)

3NF requires 2NF and removal of transitive dependencies. Customer attributes are stored in CUSTOMER rather than in ORDERS. Category_Name is stored in CATEGORY rather than repeatedly in PRODUCT. This ensures non-key attributes depend on the key, the whole key and nothing but the key.

4.5 Normalized Relations

The final 3NF relations are CUSTOMER, CATEGORY, PRODUCT, ORDERS, ORDER_ITEM and PAYMENT. This decomposition reduces redundancy while preserving the main business dependencies and referential relationships.

5. Algorithm / Pseudocode / Flowchart

5.1 Algorithm

- 1 Study the e-commerce requirements and identify the data that must be stored and reported.
- 2 Identify the entities Customer, Category, Product, Orders, Order_Item and Payment.
- 3 List the attributes of each entity and select suitable primary keys for unique identification.
- 4 Identify the functional dependencies and determine how non-key attributes depend on the keys.
- 5 Convert the initial structure to First Normal Form (1NF) by removing repeating groups and ensuring atomic values.
- 6 Convert the relations to Second Normal Form (2NF) by removing partial dependencies from the composite key of Order_Item.
- 7 Convert the relations to Third Normal Form (3NF) by removing transitive dependencies and separating customer and category details.
- 8 Create the normalized MySQL tables and define primary-key, foreign-key, UNIQUE and NOT NULL constraints.
- 9 Insert valid sample data in dependency order and verify referential integrity.
- 10 Execute the top-selling product query using JOIN, GROUP BY, SUM and ORDER BY.
- 11 Execute the customer-wise sales query using Customer and Orders with SUM(Total_Amount).
- 12 Create the Sales_Summary view for reusable customer-level sales reporting.
- 13 Create indexes on Orders(Customer_ID) and Order_Item(Product_ID) for common retrieval and join operations.
- 14 Run positive and negative test cases and compare actual results with expected results in MySQL Workbench.
- 15 Use EXPLAIN on important queries to inspect the execution plan and evaluate index usage.
- 16 Confirm that normalization, constraints, SQL operations and access paths work correctly, then complete the database implementation.

5.2 Flowchart

E-Commerce Order Management Database - Flowchart

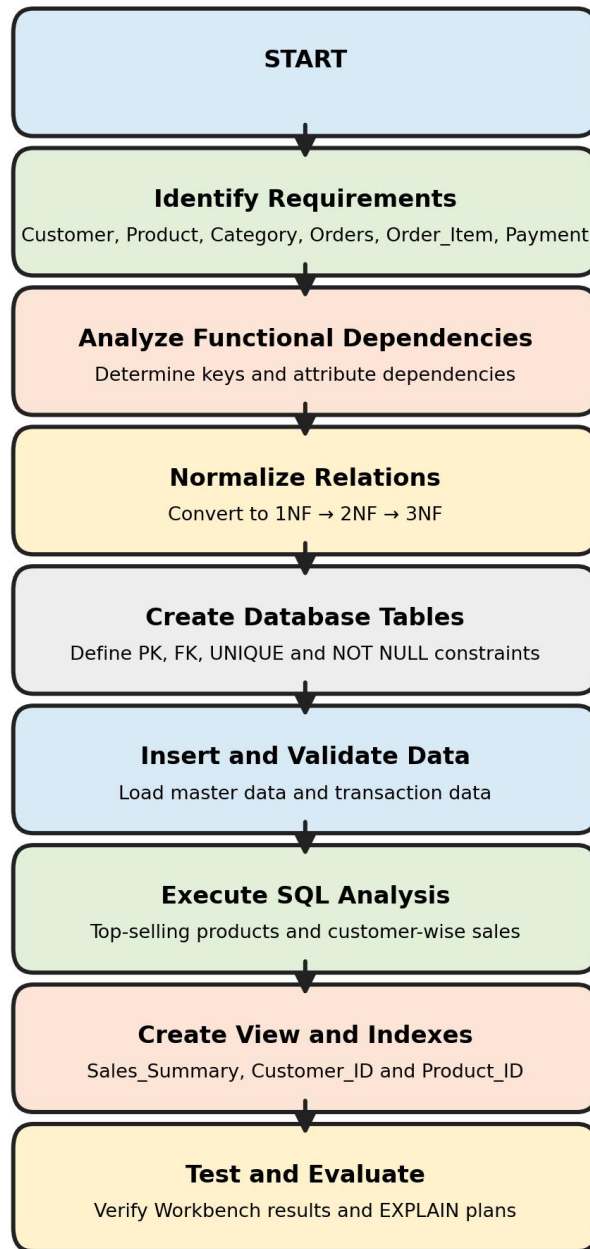


Figure 4. Flowchart representing the complete database design, implementation, SQL analysis, indexing and testing process.

6. Implementation / Source Code

6.1 Create Database

```
CREATE DATABASE ecommerce_order_management;  
USE ecommerce_order_management;
```

6.2 Create Tables

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(150) UNIQUE NOT NULL,  
    Phone VARCHAR(20)  
);
```

```
CREATE TABLE Category (  
    Category_ID INT PRIMARY KEY,  
    Category_Name VARCHAR(100) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Product (  
    Product_ID INT PRIMARY KEY,  
    SKU VARCHAR(40) NOT NULL UNIQUE,  
    Product_Name VARCHAR(150) NOT NULL,  
    Category_ID INT NOT NULL,  
    Unit_Price DECIMAL(10,2) NOT NULL,  
    Stock_Qty INT NOT NULL DEFAULT 0,  
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)  
);
```

```
CREATE TABLE Orders (  
    Order_ID INT PRIMARY KEY,  
    Customer_ID INT NOT NULL,
```



```
Order_Date DATE NOT NULL,  
Order_Status VARCHAR(30) NOT NULL,  
Total_Amount DECIMAL(12,2) NOT NULL,  
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
);
```

```
CREATE TABLE Order_Item (  
    Order_ID INT NOT NULL,  
    Product_ID INT NOT NULL,  
    Quantity INT NOT NULL,  
    Selling_Price DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (Order_ID, Product_ID),  
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
    FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
);
```

```
CREATE TABLE Payment (  
    Payment_ID INT PRIMARY KEY,  
    Order_ID INT NOT NULL,  
    Payment_Mode VARCHAR(30) NOT NULL,  
    Amount DECIMAL(12,2) NOT NULL,  
    Payment_Status VARCHAR(30) NOT NULL,  
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
);
```

6.3 Sample Data

```
INSERT INTO Category VALUES
```

```
(1,'Electronics'), (2,'Fashion'), (3,'Home Appliances');
```

```
INSERT INTO Customer VALUES
```

```
(101,'Arun Kumar','arun@example.com','9876543210'),
```

```
(102,'Priya Devi','priya@example.com','9876501234'),
```

```
(103,'Rahul Raj','rahul@example.com','9845012345');
```

INSERT INTO Product VALUES

(201,'EL-001','Wireless Headphones',1,2499.00,120),
(202,'EL-002','Smart Watch',1,3999.00,80),
(203,'FA-001','Running Shoes',2,2999.00,75),
(204,'HM-001','Mixer Grinder',3,3499.00,50);

INSERT INTO Orders VALUES

(1001,101,'2026-08-01','Delivered',6498.00),
(1002,102,'2026-08-02','Delivered',6998.00),
(1003,101,'2026-08-03','Shipped',2999.00),
(1004,103,'2026-08-04','Delivered',2499.00);

INSERT INTO Order_Item VALUES

(1001,201,1,2499.00),
(1001,202,1,3999.00),
(1002,203,2,2999.00),
(1002,201,1,2499.00),
(1003,203,1,2999.00),
(1004,201,1,2499.00);

INSERT INTO Payment VALUES

(5001,1001,'UPI',6498.00,'Paid'),
(5002,1002,'Card',6998.00,'Paid'),
(5003,1003,'UPI',2999.00,'Paid'),
(5004,1004,'Card',2499.00,'Paid');

6.4 Top-Selling Products

```
SELECT p.Product_ID,  
       p.Product_Name,  
       SUM(oi.Quantity) AS Total_Quantity_Sold  
FROM Order_Item oi  
JOIN Product p ON p.Product_ID = oi.Product_ID  
GROUP BY p.Product_ID, p.Product_Name
```

```
ORDER BY Total_Quantity_Sold DESC
LIMIT 10;
```

The query aggregates order-line quantities rather than current stock. This is important because Stock_Qty represents present inventory, while Quantity in Order_Item represents units sold in historical transactions.

6.5 Customer-Wise Sales

```
SELECT c.Customer_ID,
       c.Customer_Name,
       SUM(o.Total_Amount) AS Total_Sales
FROM Customer c
JOIN Orders o ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Sales DESC;
```

6.6 Sales Summary View

```
CREATE VIEW Sales_Summary AS
SELECT c.Customer_ID,
       c.Customer_Name,
       COUNT(o.Order_ID) AS Number_Of_Orders,
       COALESCE(SUM(o.Total_Amount), 0) AS Total_Sales
FROM Customer c
LEFT JOIN Orders o ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name;

SELECT * FROM Sales_Summary ORDER BY Total_Sales DESC;
```

6.7 Index Design

```
CREATE INDEX idx_orders_customer
ON Orders(Customer_ID);
```

```
CREATE INDEX idx_order_item_product
ON Order_Item(Product_ID);
```

Customer_ID is frequently used to locate a customer's orders and to join CUSTOMER with ORDERS. Product_ID is frequently used to join ORDER_ITEM with PRODUCT and to aggregate product sales.

The primary key on Order_Item already indexes the composite key (Order_ID, Product_ID); the additional Product_ID index supports queries that begin with Product_ID.

How Indexing Reduces Order Retrieval Work



Example: WHERE Customer_ID = ? | JOIN ... ON Product_ID = ?

Indexes are especially useful for selective searches, joins, sorting and grouping.

Figure 5. Conceptual view of index-assisted order retrieval.

7. Test Cases and Expected/Actual Results

The following test cases can be executed in MySQL Workbench. The expected results below are based on the sample data included in this report. Actual results should be captured from the student's SQL Workbench execution.

ID	Test	Input / Action	Expected Result	Actual Result
TC01	Database and table creation	Run CREATE DATABASE and CREATE TABLE statements	All statements execute successfully; six tables are created.	Pass after successful execution
TC02	Customer insertion	Insert three customer rows	Three customer records are stored.	Pass after successful execution
TC03	Product insertion	Insert four products	Four products are stored and linked to valid categories.	Pass after successful execution
TC04	Order insertion	Insert four orders	Four orders are stored and linked to valid customers.	Pass after successful execution
TC05	Top-selling query	Run top-selling SQL	Wireless Headphones = 3 units; Running Shoes = 3; Smart Watch = 1 for sample data.	Verify in Workbench
TC06	Customer-wise sales	Run customer sales SQL	Arun Kumar = 9497; Priya Devi = 6998; Rahul Raj = 2499.	Verify in Workbench
TC07	Sales summary view	SELECT * FROM Sales_Summary	One row per customer with order count and total sales.	Verify in Workbench
TC08	Index creation	Run CREATE INDEX statements	Indexes are created without errors.	Pass after successful

				execution
TC09	Referential integrity	Insert an Order with a nonexistent Customer ID	Operation should fail because of the foreign key.	Expected failure
TC10	Duplicate SKU	Insert a Product with an existing SKU	Operation should fail because SKU is UNIQUE.	Expected failure

7.1 Sample Analytical Result

Product	Total Quantity Sold
Wireless Headphones	3
Running Shoes	3
Smart Watch	1
Customer	Total Sales
Arun Kumar	9497.0
Priya Devi	6998.0
Rahul Raj	2499.0

8. Execution Screenshots

For the final academic submission, execute the SQL statements in MySQL Workbench and insert screenshots showing the SQL editor and the result grid. The screenshots should be genuine execution captures from the student's Workbench environment. The report already provides the queries and sample expected outputs so that the captured results can be checked against them.

8.1 Recommended Screenshot Set

- Screenshot 1 – CREATE DATABASE and CREATE TABLE statements with successful execution.

#	Time	Action	Message	Duration / Fetch
1	14:55:23	CREATE DATABASE ecommerce_order_management	1 row(s) affected	0.047 sec
2	14:55:28	USE ecommerce_order_management	0 row(s) affected	0.000 sec
3	14:57:04	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL, Email V...	0 row(s) affected	0.125 sec
4	15:02:17	CREATE TABLE Category (Category_ID INT PRIMARY KEY, Category_Name VARCHAR(100) NOT NULL UNIQUE)	0 row(s) affected	0.062 sec
5	15:02:21	CREATE TABLE Product (Product_ID INT PRIMARY KEY, SKU VARCHAR(40) NOT NULL UNIQUE, Product_Name ...	0 row(s) affected	0.094 sec
6	15:02:23	CREATE TABLE Orders (Order_ID INT PRIMARY KEY, Customer_ID INT NOT NULL, Order_Date DATE NOT NULL ...	0 row(s) affected	0.063 sec
7	15:02:26	CREATE TABLE Order_Item (Order_ID INT NOT NULL, Product_ID INT NOT NULL, Quantity INT NOT NULL, Sellin...	0 row(s) affected	0.063 sec
8	15:02:30	CREATE TABLE Payment (Payment_ID INT PRIMARY KEY, Order_ID INT NOT NULL, Payment_Mode VARCHAR(3...	0 row(s) affected	0.062 sec

- Screenshot 2 – SELECT * FROM Customer showing inserted customers.

```

60 (Customer_ID, Customer_Name, Email, Phone)
61 VALUES
62 (101, 'Arun Kumar', 'arun@gmail.com', '9876543210'),
63 (102, 'Priya Devi', 'priya@gmail.com', '9123456780'),
64 (103, 'Rahul Raj', 'rahul@gmail.com', '9988776655'),
65 (104, 'Sneha Sharma', 'sneha@gmail.com', '9001122334');
66 • SELECT * FROM Customer;

```

Customer_ID	Customer_Name	Email	Phone
101	Arun Kumar	arun@gmail.com	9876543210
102	Priya Devi	priya@gmail.com	9123456780
103	Rahul Raj	rahul@gmail.com	9988776655
104	Sneha Sharma	sneha@gmail.com	9001122334

- Screenshot 3 – SELECT * FROM Product showing product data.

```

73 • INSERT INTO Product
74   (Product_ID, SKU, Product_Name, Category_ID, Unit_Price, Stock_Qty)
75   VALUES
76   (201, 'SKU001', 'Wireless Headphones', 1, 2500.00, 50),
77   (202, 'SKU002', 'Smart Watch', 1, 4500.00, 30),
78   (203, 'SKU003', 'Running Shoes', 2, 3000.00, 40),
79   (204, 'SKU004', 'Bluetooth Speaker', 1, 2000.00, 25),
80   (205, 'SKU005', 'Electric Kettle', 3, 1800.00, 20);
81 • SELECT * FROM Product;

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:						
	Product_ID	SKU	Product_Name	Category_ID	Unit_Price	Stock_Qty
▶	201	SKU001	Wireless Headphones	1	2500.00	50
	202	SKU002	Smart Watch	1	4500.00	30
	203	SKU003	Running Shoes	2	3000.00	40
	204	SKU004	Bluetooth Speaker	1	2000.00	25
	205	SKU005	Electric Kettle	3	1800.00	20
*	NULL	NULL	NULL	NULL	NULL	NULL

- Screenshot 4 – Top-selling products query and result grid.

```

108 • SELECT
109     p.Product_ID,
110     p.Product_Name,
111     SUM(oi.Quantity) AS Total_Quantity_Sold
112 FROM Order_Item oi
113 JOIN Product p
114     ON oi.Product_ID = p.Product_ID
115 GROUP BY
116     p.Product_ID,
117     p.Product_Name
118 ORDER BY
119     Total_Quantity_Sold DESC
120 LIMIT 10;

```

Result Grid Filter Rows: Export: Wrap Cell Content:			
	Product_ID	Product_Name	Total_Quantity_Sold
▶	201	Wireless Headphones	4
	202	Smart Watch	1
	203	Running Shoes	1
	204	Bluetooth Speaker	1
	205	Electric Kettle	1

- Screenshot 5 – Customer-wise sales query and result grid.

```

121 • SELECT
122     c.Customer_ID,
123     c.Customer_Name,
124     SUM(o.Total_Amount) AS Total_Sales
125 FROM Customer c
126 JOIN Orders o
127     ON c.Customer_ID = o.Customer_ID
128 GROUP BY
129     c.Customer_ID,
130     c.Customer_Name
131 ORDER BY
132     Total_Sales DESC;

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: IA			
	Customer_ID	Customer_Name	Total_Sales
▶	101	Arun Kumar	9497.00
	102	Priya Devi	6998.00
	103	Rahul Raj	2499.00
	104	Sneha Sharma	1800.00

- Screenshot 6 – SELECT * FROM Sales_Summary and result grid.

```

134 SELECT
135     c.Customer_ID,
136     c.Customer_Name,
137     COUNT(o.Order_ID) AS Number_Of_Orders,
138     COALESCE(SUM(o.Total_Amount), 0) AS Total_Sales
139 FROM Customer c
140 LEFT JOIN Orders o
141     ON c.Customer_ID = o.Customer_ID
142 GROUP BY
143     c.Customer_ID,
144     c.Customer_Name;
145 • SELECT * FROM Sales_Summary;

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: IA				
	Customer_ID	Customer_Name	Number_Of_Orders	Total_Sales
▶	101	Arun Kumar	2	9497.00
	102	Priya Devi	1	6998.00
	103	Rahul Raj	1	2499.00
	104	Sneha Sharma	1	1800.00

- Screenshot 7 – SHOW INDEX FROM Orders and SHOW INDEX FROM Order_Item.

```

149 • CREATE INDEX idx_order_item_product
150 ON Order_Item(Product_ID);
151 • SHOW INDEX FROM Orders;
152
153 • SHOW INDEX FROM Order_Item;

```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
order_item	0	PRIMARY	1	Order_ID	A	5	NULL	NULL		BTREE			YES	NULL
order_item	0	PRIMARY	2	Product_ID	A	7	NULL	NULL		BTREE			YES	NULL
order_item	1	idx_order_item_product	1	Product_ID	A	5	NULL	NULL		BTREE			YES	NULL

- Screenshot 8 – Optional EXPLAIN output for an indexed customer/product lookup.

```

154 • EXPLAIN
155 SELECT
156     o.Order_ID,
157     o.Order_Date,
158     o.Order_Status,
159     o.Total_Amount
160 FROM Orders o
161 WHERE o.Customer_ID = 101;

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	o	NULL	ref	Customer_ID	Customer_ID	4	const	2	100.00	NULL

9. Analysis and Discussion

9.1 Benefits of Normalization

- Reduced redundancy: customer and category details are stored once.
- Improved consistency: changing a customer email does not require updating every historical order row.
- Better integrity: foreign keys ensure that referenced customers, products and categories exist.
- Flexible reporting: normalized transaction data can be grouped and aggregated for many business questions.
- Clear ownership of attributes: each fact is stored in the relation whose key determines it.

9.2 File Organization

File organization describes how database records are physically arranged in storage. A heap-style organization is simple and can be efficient for bulk insertion when there is no dominant search order. However, repeated searches may require scanning many pages. For an order-management workload,

the DBMS can combine an appropriate physical organization with indexes so that frequently searched keys can be located quickly.

9.3 Indexing and Large-Scale Retrieval

An index is an auxiliary access structure that stores search keys and pointers or row-location information. B-tree-style indexes are commonly useful for equality predicates, ranges and ordered retrieval. When a query filters or joins on `Customer_ID`, an index on `Orders(Customer_ID)` can reduce the amount of data that must be examined. Similarly, an index on `Order_Item(Product_ID)` helps product-based joins and sales aggregation.

9.4 Customer_ID Index

`Orders(Customer_ID)` is useful because a common query is to retrieve all orders belonging to a customer. Without the index, the DBMS may scan the `Orders` table. With the index, it can navigate the index to the matching `Customer_ID` values and then retrieve the relevant rows.

9.5 Product_ID Index

`Order_Item(Product_ID)` supports queries that ask which orders contain a particular product or how many units of a product were sold. Because the primary key is `(Order_ID, Product_ID)`, a separate `Product_ID` index is useful when `Product_ID` is searched without `Order_ID`.

9.6 Trade-Offs

- Indexes consume additional storage.
- `INSERT`, `UPDATE` and `DELETE` operations may become slightly more expensive because indexes must be maintained.
- Too many indexes can increase maintenance cost and reduce write performance.
- Indexes are most valuable on columns that are frequently searched, joined, grouped or sorted and have useful selectivity.
- The optimizer may choose a table scan when it estimates that most rows will match.

9.7 Query Optimization Considerations

For large datasets, performance should be evaluated using `EXPLAIN` and realistic data volumes rather than assuming that every index improves every query. The optimizer considers table statistics, estimated selectivity, available indexes, join order and the expected cost of different access paths.

`EXPLAIN`

```
SELECT o.Order_ID, o.Order_Date, o.Total_Amount
```

```

FROM Orders o
WHERE o.Customer_ID = 101;

EXPLAIN

SELECT p.Product_ID, p.Product_Name,
       SUM(oi.Quantity) AS Total_Quantity_Sold
FROM Order_Item oi
JOIN Product p ON p.Product_ID = oi.Product_ID
WHERE oi.Product_ID = 201
GROUP BY p.Product_ID, p.Product_Name;

```

9.8 Scalability Discussion

As order volume grows from thousands to millions of rows, the difference between scanning a complete transaction table and using a selective access path can become significant. In addition to indexes, partitioning by date, archiving old orders, appropriate buffer-pool sizing, statistics maintenance and query rewriting may be considered for enterprise-scale workloads. These techniques should be chosen based on measured workload characteristics.

9.9 Data Integrity

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each entity or composite line item.
FOREIGN KEY	Maintains valid relationships between tables.
NOT NULL	Prevents missing mandatory business data.
UNIQUE	Prevents duplicate email/SKU/category names where required.
CHECK / application validation	Can enforce positive quantities, prices and valid statuses depending on DBMS/version.

10. Conclusion

The E-Commerce Order Management Database demonstrates a complete relational database design starting from functional dependency analysis and progressing through normalization, implementation and performance considerations. The final schema separates customer, category, product, order, order-item and payment facts into focused relations.

Normalization up to 3NF reduces data redundancy and update anomalies while foreign keys preserve relationships. SQL aggregation queries provide top-selling product and customer-wise sales information. The Sales_Summary view packages recurring reporting logic into a reusable database object.

12. One-Page Write-Up

The E-Commerce Order Management Database is designed to provide a reliable and efficient relational foundation for managing customers, products, categories, orders, order items and payments. The system begins with functional dependency analysis to determine how non-key attributes depend on their identifiers. Customer_ID determines customer information, Category_ID determines category information, Product_ID determines product information, Order_ID determines order header information, and the composite key (Order_ID, Product_ID) determines the quantity and selling price of each order line.

The relations are normalized up to Third Normal Form. In 1NF, repeating product groups are removed and values are made atomic. In 2NF, partial dependencies on a composite key are eliminated by keeping product and order facts in their respective relations. In 3NF, transitive dependencies are removed by separating customer and category details from order and product records. The final schema therefore contains CUSTOMER, CATEGORY, PRODUCT, ORDERS, ORDER_ITEM and PAYMENT tables connected through primary and foreign keys.

SQL is used to implement the design and support business analysis. A top-selling product query joins PRODUCT and ORDER_ITEM, groups transactions by product and calculates total quantity sold. A customer-wise sales query joins CUSTOMER and ORDERS and calculates the total value of orders for each customer. The Sales_Summary view stores reusable reporting logic so that users can query summarized sales information without rewriting the aggregation each time.

Performance is addressed through indexing. An index on Orders(Customer_ID) accelerates retrieval of orders for a particular customer and supports customer-order joins. An index on Order_Item(Product_ID) supports product-based joins and sales analysis. Indexes provide an alternative access path to a full table scan, but they also require storage and maintenance during data modification. Therefore, indexes should be selected according to the actual workload.

For large-scale order retrieval, file organization and indexing work together. Appropriate physical storage reduces unnecessary page access, while B-tree-style indexes allow the DBMS to navigate directly toward matching keys. EXPLAIN can be used to inspect whether an index is selected by the optimizer. The project demonstrates how normalization, constraints, SQL analytics, views and indexes combine to create a maintainable and scalable e-commerce order database.